Recursion vs. Iteration

Recursive vs. Iterative Approach: Exploring the Power of Recursion

Recursion and iteration are two fundamental concepts in programming that enable us to solve problems through repetitive actions. While both approaches involve repetition, they differ in their implementation and mindset. The following table makes a comprehensive comparison to state how recursive and iterative thinking differ from each other.

No.	Recursive Thinking	Iterative Thinking
1.	Breaks down a complex problem into **smaller**, similar **subproblems**.	Solves a problem through **repetitive execution** of a set of instructions.
2.	Involves solving the **base case(s)** and recursively solving smaller instances of the problem.	Utilizes **loops** to repeatedly execute a block of code until a specific condition is met.
3.	Often **mirrors** the natural way we think about and solve problems.	Relies on **iteration** and repetition to reach a solution.
4.	Can be more intuitive and **concise** for problems with a recursive structure.	Can be more straightforward and efficient for problems with a **sequential** nature.
5.	Emphasizes **problem decomposition** and subproblem relationships.	Focuses on **step-by-step** execution and maintaining state.
6.	Can lead to elegant and **readable code**, especially for problems with inherent recursive patterns.	Can be **efficient** in terms of time and space complexity for problems with a **linear structure**.
7.	Can be suitable for problems with **dynamic** or changing input sizes.	Well-suited for problems that require repetitive operations or iteration **over a collection**.
8.	May have **higher memory usage** due to recursive function calls and maintaining call stack.	Typically has **lower memory usage** as it avoids recursive function calls.
9.	Can be more **abstract** and focused on the problem's essence rather than implementation details.	Often requires **explicit** handling of loop variables and maintaining program state.

The comparison presented in the above table highlights the strengths and weaknesses of recursive and iterative solutions. The choice between a recursive or iterative solution depends on the problem, its requirements, and factors like performance, simplicity, and readability.

Sometimes, keeping the perks of a particular approach in view, you may want to translate a recursive solution to an iterative equivalent or vice versa. In the upcoming sections, we discuss the systematic approaches to do these translations.

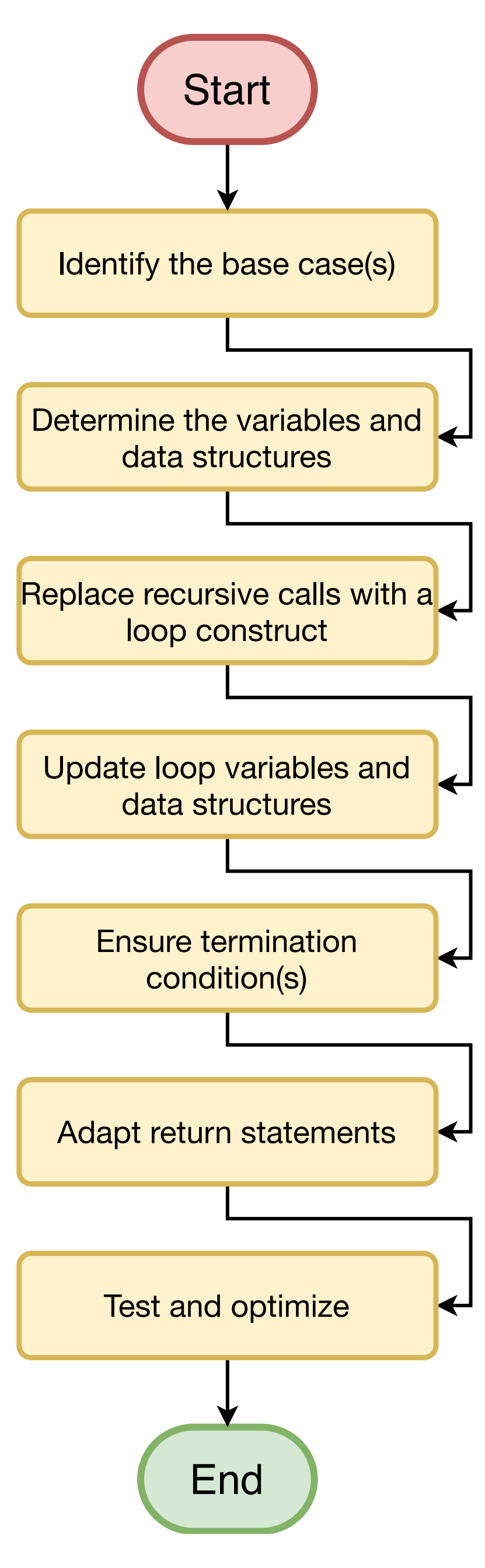
Converting a Recursive Solution to an Iterative Equivalent

Translating an iterative solution to an equivalent parallel algorithm that specifically runs on GPU-based massively parallel infrastructure is generally easy. It is evident by the fact that older GPUs lack the ability to handle recursive functions. Thereby, before designing the parallel counterpart, you may prefer translating the recursive solution to the iterative one.

There is no formally recognized approach to realize the conversion. However, here is a widely-accepted systematic approach to convert a recursive solution to an iterative equivalent:

1. Identify the base case(s).
2. Determine the variables and data structures.
3. Replace recursive calls with a loop construct.
4. Update loop variables and data structures.
5. Ensure termination condition(s).
6. Adapt return statements.
7. Test and optimize.

All the above-mentioned steps can be depicted as a flowchart diagram as given below.


Start
Identify the base case(s)
Determine the variables and data structures
Replace recursive calls with a loop construct
Update loop variables and data structures
Ensure termination condition(s)
Adapt return statements
Test and optimize
End

Each step focuses on a specific aspect of the conversion process, allowing you to systematically convert the recursive solution to an iterative one.

Example

Let's look at an example where we convert a recursive factorial calculation algorithm to an iterative one. The pseudocode of the recursive solution is as follows:

```
function factorial(n):
    // Base case
    if n == 0 or n == 1:
        return 1
    // Recursive case
    else:
        return n * factorial(n - 1)
```

Let's follow our systematic framework to realize this recursive to iterative conversion.

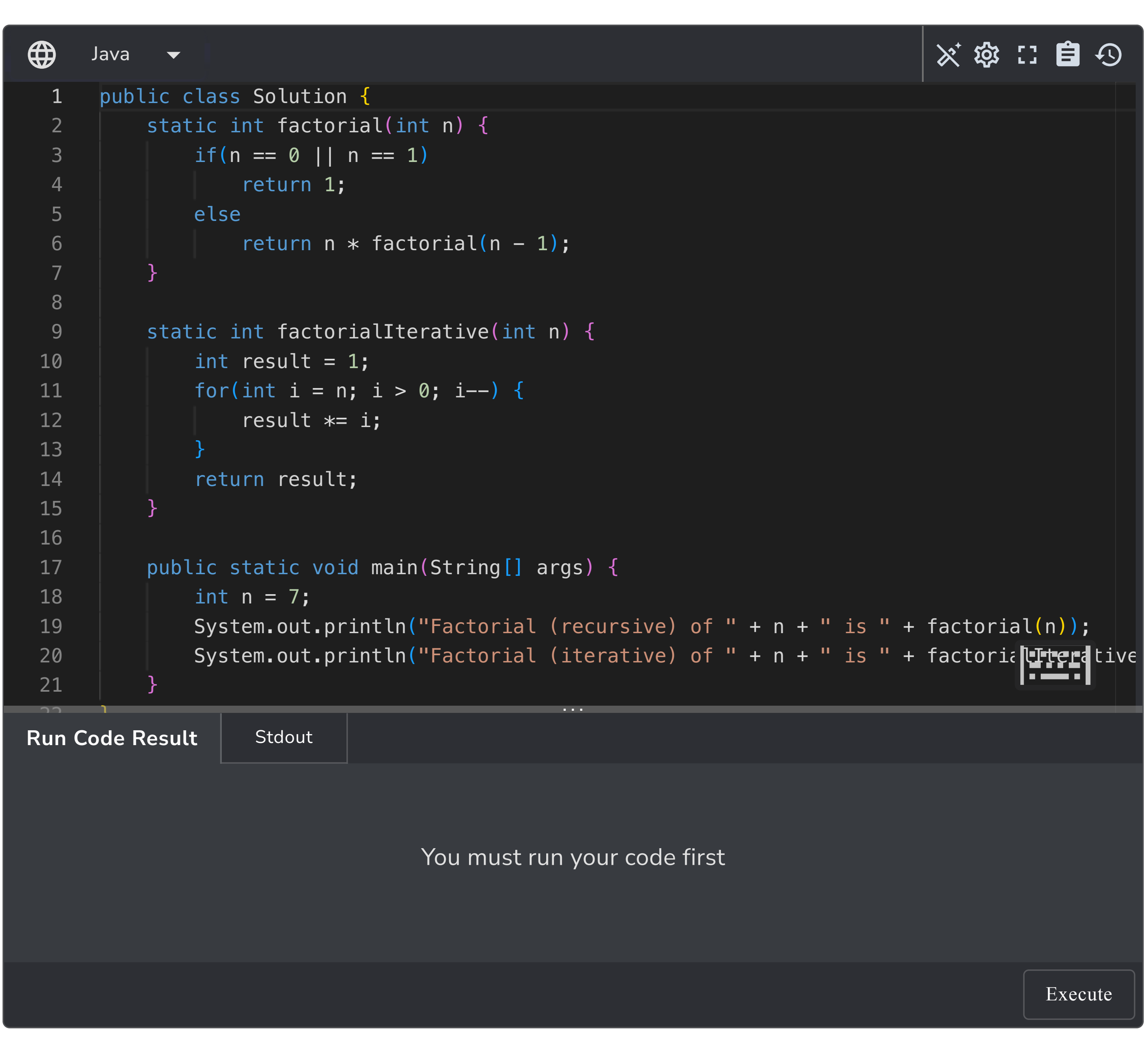
1. **Identify the base case(s)**: Look for the condition(s) in the recursive solution that determines when the recursion should stop. In the factorial example, the base case is when `n` is `0` or `1`. At the base cases, the factorial value is `1`.
2. **Determine the variables and data structures**: Identify the variables and data structures needed to track the state of the computation. In the factorial example, we only need a single variable `result` to store the factorial value.
3. **Replace recursive calls with a loop construct**: Instead of making recursive calls, use a loop construct (such as a `for` loop or a `while` loop) to iterate over the necessary range. We can add a `for` loop from `n` down to the base case `1`.
4. **Update loop variables and data structures**: Adjust the loop variables and data structures to simulate the progress of the recursion. In the factorial example, the loop variable `i` represents the decreasing value of `n` in each iteration.
5. **Ensure termination condition(s)**: Make sure the termination condition(s) in the recursive solution corresponds to the loop condition(s) in the iterative solution. In the factorial example, the loop iterates until `i` reaches base case `1`.
6. **Adapt return statements**: Modify any return statements in the recursive solution to set the final result in the iterative solution. In the factorial example, we return the `result` variable after the loop completes.
7. **Test and optimize**: Test the iterative solution to ensure its correctness and compare its performance to the recursive solution. You can further optimize the iterative solution based on the specific requirements of the problem and the available optimization techniques (e.g., loop unrolling).

So, here is the final outcome, which is an iterative solution to calculate the factorial.

```
function factorialIterative(n):
    result = 1
    for i from n down to 1:
        result = result * i
    return result
```

This iterative solution uses a loop to multiply numbers from `n` down to `1` and updates the result variable in each iteration. Once `i` reaches `1`, we become sure that we have multiplied all the numbers required for factorial calculation (i.e., $n \times n-1 \times n-2 \times n-3 \times \ldots \times 1$) and have aggregated the right term in the `result` variable. Thereby, we return it then. For the input `0`, we expect the loop condition to fail, so the `result` stores and returns `1` in this case.

Here is the complete working code for the recursive and iterative factorial calculation solutions.



```java
public class Solution {
    static int factorial(int n) {
        if(n == 0 || n == 1)
            return 1;
        else
            return n * factorial(n - 1);
    }

    static int factorialIterative(int n) {
        int result = 1;
        for(int i = n; i > 0; i--) {
            result *= i;
        }
        return result;
    }

    public static void main(String[] args) {
        int n = 7;
        System.out.println("Factorial (recursive) of " + n + " is " + factorial(n));
        System.out.println("Factorial (iterative) of " + n + " is " + factorialIterative
    }
}
```

You must run your code first

Great, that was easy. The approach works for most of the conversions; however, complex conversions may take several iterations to succeed.

Point to Pounder: Does every recursive solution has an iterative equivalent? The answer is **yes**, but **why**?

Here's the reason why: recursion and iteration are both programming constructs that allow you to repeat certain instructions or steps. Recursion does this by having a function call itself until a base case is reached, while iteration does it with looping constructs like `for`, `while`, and `do-while` loops.

To convert a recursive solution to an iterative one, you generally need to manage the state that would normally be maintained by the recursion stack yourself. This often involves using your own stack or queue data structure.

Let's move on to the next section, which discusses converting an iterative solution to a recursive one.

Converting an Iterative Solution to a Recursive Equivalent

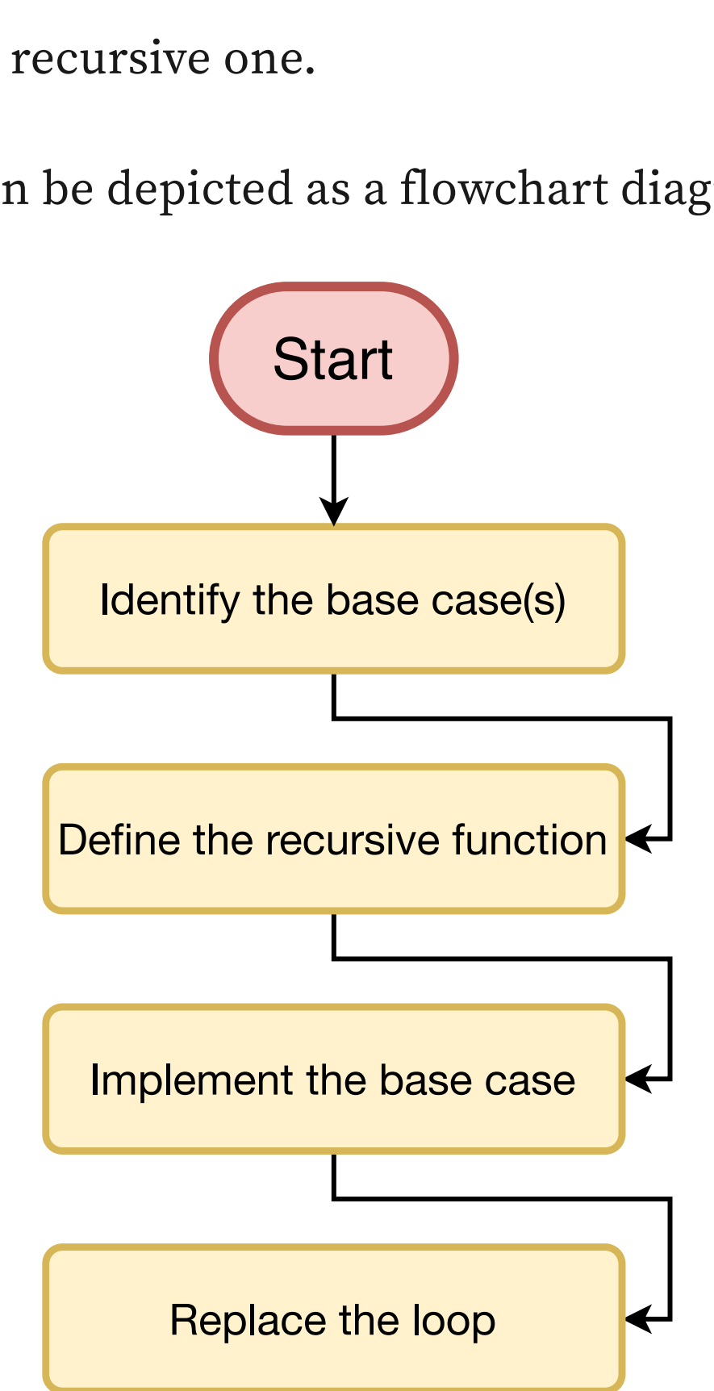
Iterative solutions are often preferred due to their good performance and lower resource utilization. However, sometimes, you may require the recursive solution to make the solution more readable, elegant, and natural.

Here are the steps to convert an iterative solution to a recursive solution:

1. Identify the base case.
2. Define the recursive function.
3. Implement the base case.
4. Replace the loop
5. Return the result

Each step focuses on a specific aspect of the conversion process, allowing you to systematically convert the iterative solution to a recursive one.

All the above-mentioned steps can be depicted as a flowchart diagram as given below.



Example

Assume you need to convert the following iterative factorial calculation algorithm to a recursive counterpart.

```
function factorialIterative(n):
    result = 1
    for i from n down to 1:
        result = result * i
    return result
```

After applying the above-mentioned conversion steps, we can convert this iterative solution to the recursive solution:

1. **Identify the base case**:
   - In the iterative solution, the base case is when `i` reaches `1`. Moreover, for input `0`, the function returns `1`. Therefore, our base cases are input values `0` and `1`.
2. **Define the recursive function**:
   - Create a function called `factorialRecursive` that takes `n` as a parameter.
3. **Implement the base case**:
   - Check if `n` equals `0` or `1`; if so, return `1`.
4. **Replace the loop**:
   - Instead of the loop, make a recursive call to `factorialRecursive` with `n - 1` as the parameter, imitating the decrement in each loop iteration.
5. **Return the result**:
   - Return the result of the recursive call multiplied by `n`.

Here is the pseudocode of the resultant recursive solution.

```
function factorialRecursive(n):
    if n == 0 or n==1:
        return 1
    else:
        return n * factorialRecursive(n - 1)
```

Challenge: Write the iterative and recursive functions for calculating the sum of odd numbers between `0` and a given number `n`.